


**UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO**

**Facultad de Ciencias**


# Practica 2

*Analizadores léxicos con Lex (Flex)*


| No. de cuenta | Nombre |
|---|---|
| **321225087** | Rivera Jimenez Isaac |




**Compiladores**

Semestre 2027-1

# Objetivo

Que el alumno conozca e implemente los principios del análisis sintáctico descendente recursivo, aplicando técnicas de transformación de gramáticas —eliminación de ambigüedad, eliminación de recursividad izquierda y factorización izquierda— para construir un analizador sintáctico predictivo que, junto con el analizador léxico desarrollado con Flex, permita reconocer programas válidos del lenguaje $C_1$.

# Ejercicios

Dada la gramática $G = (N, \Sigma, P, S)$, descrita por las siguientes producciones:

$$P = \begin{cases} \text{programa} & \rightarrow \text{declaraciones sentencias} \\ \text{declaraciones} & \rightarrow \text{declaraciones declaracion} \mid \text{declaracion} \\ \text{declaracion} & \rightarrow \text{tipo lista_var} \ ; \\ \text{tipo} & \rightarrow \mathbf{int} \mid \mathbf{float} \\ \text{lista_var} & \rightarrow \text{lista_var} \ , \ \text{identificador} \mid \text{identificador} \\ \text{sentencias} & \rightarrow \text{sentencias sentencia} \mid \text{sentencia} \\ \text{sentencia} & \rightarrow \text{identificador} = \text{expresion} \ ; \\ & \quad \mid \mathbf{if} \ ( \ \text{expresion} \ ) \ \text{sentencias} \ \mathbf{else} \ \text{sentencias} \\ & \quad \mid \mathbf{while} \ ( \ \text{expresion} \ ) \ \text{sentencias} \\ \text{expresion} & \rightarrow \text{expresion} + \text{expresion} \mid \text{expresion} - \text{expresion} \\ & \quad \mid \text{expresion} * \text{expresion} \mid \text{expresion} \ / \ \text{expresion} \\ & \quad \mid \text{identificador} \mid \text{numero} \mid ( \ \text{expresion} \ ) \end{cases}$$

1. Determinar en un archivo Readme, en formato Markdown (`.md`) o LaTeX (`.tex`) —con su respectivo PDF, para este último—, los conjuntos $N$, $\Sigma$ y el símbolo inicial $S$. **(0.5 pts.)**

   El conjunto $\sum$ = {*if, while, int, float, identificador, numero, else, ;, *, +, -, /, , , (,),* = }, $N$ = {programa, declaraciones, sentencias, tipo, lista_var, declaracion, sentencia, expresion}, $S = programa$ simbolo inicial.

2. Mostrar en el archivo el proceso de eliminación de ambigüedad o justificar, en caso de no ser necesario. **(1 pt.)**

   Decimos que existe ambigüedad en una gramática $G$, si es que se puede formar una cadena de diferentes formas gracias a las reglas de la gramática, para este caso la gramática, utiliza símbolos no terminales para poder realizar las producciones consecuentes de el flujo de un programa, pero algunos de los símbolos no terminales genera una misma producción o un mismo símbolo terminal, por lo tanto en la primera parte de sentencias y declaraciones no existe ambigüedad para la gramática P. Sin embargo para el símbolo no terminal *expresión* se puede generar una misma expresión con dos reglas de la gramática, como por ejemplo, para la cadena `id + id * id`, la regla permite generar dos interpretaciones gramaticales válidas:

   *a*) `(id + id) * id` (Evalúa primero la suma).

   *b*) `id + (id * id)` (Evalúa primero la multiplicación).

   Por lo tanto para eliminar la ambigüedad, se separan los operadores en niveles jerárquicos introduciendo dos nuevos símbolos no terminales: `termino` y `factor`, así.

```
E -> E + T | E- T| T
T -> T*F | T / F | F
F -> (E) | id | num
```

Transportandolo a la gramatica P tenemos:

$$\begin{aligned} \text{expresion} &\rightarrow \text{expresion} + \text{termino} \mid \text{expresion} - \text{termino} \mid \text{termino} \\ \text{termino} &\rightarrow \text{termino} * \text{factor} \mid \text{termino} / \text{factor} \mid \text{factor} \\ \text{factor} &\rightarrow ( \text{ expresion } ) \mid \mathbf{identificador} \mid \mathbf{numero} \end{aligned}$$

3. Mostrar en el archivo el proceso de eliminación de la recursividad izquierda o justificar, en caso de no ser necesario. **(1 pt.)**

   Existen varias producciones no terminales que se llaman recursivamente, tal es el caso de:

$$\begin{cases} \text{declaraciones} & \rightarrow \text{declaraciones declaracion} \mid \text{declaracion} \\ \text{lista_var} & \rightarrow \text{lista_var} \ , \ \mathbf{identificador} \mid \mathbf{identificador} \\ \text{sentencias} & \rightarrow \text{sentencias sentencia} \mid \text{sentencia} \\ \text{expresion} & \rightarrow \text{expresion} + \text{termino} \mid \text{expresion} - \text{termino} \mid \text{termino} \\ \text{termino} & \rightarrow \text{termino} * \text{factor} \mid \text{termino} / \text{factor} \mid \text{factor} \end{cases}$$

   Consideramos la nueva implementacion de P quitando la ambiguedad de la gramatica, se llama recursivamente, por lo tanto para poderlo quitar debemos de seguir la regla $A \rightarrow A\alpha_1|A\alpha_2...|\beta_1|\beta_2|...$, para despues: $A \rightarrow \beta_1 A'|\beta_2 A'...|\beta_n A'$, $A \rightarrow \alpha_1 A'|\alpha_2 A'|...\alpha_m A'|\epsilon$, de esta forma aplicando esta transformación a cada producción obtenemos:

   - `declaraciones` ($\alpha =$ declaracion, $\beta =$ declaracion):

$$\begin{aligned} \text{declaraciones} &\rightarrow \text{declaracion declaraciones}' \\ \text{declaraciones}' &\rightarrow \text{declaracion declaraciones}' \mid \epsilon \end{aligned}$$

   - `lista_var` ($\alpha =$ , $\mathbf{identificador}$, $\beta = \mathbf{identificador}$):

$$\begin{aligned} \text{lista_var} &\rightarrow \mathbf{identificador}\ \text{lista_var}' \\ \text{lista_var}' &\rightarrow \ , \ \mathbf{identificador}\ \text{lista_var}' \mid \epsilon \end{aligned}$$

   - `sentencias` ($\alpha =$ sentencia, $\beta =$ sentencia):

$$\begin{aligned} \text{sentencias} &\rightarrow \text{sentencia sentencias}' \\ \text{sentencias}' &\rightarrow \text{sentencia sentencias}' \mid \epsilon \end{aligned}$$

   - `expresion` ($\alpha_1 = +$ termino, $\alpha_2 = -$ termino, $\beta =$ termino):

$$\begin{aligned} \text{expresion} &\rightarrow \text{termino expresion}' \\ \text{expresion}' &\rightarrow + \text{ termino expresion}' \mid - \text{ termino expresion}' \mid \epsilon \end{aligned}$$

   - `termino` ($\alpha_1 = *$ factor, $\alpha_2 = /$ factor, $\beta =$ factor):

$$\begin{aligned} \text{termino} &\rightarrow \text{factor termino}' \\ \text{termino}' &\rightarrow * \text{ factor termino}' \mid / \text{ factor termino}' \mid \epsilon \end{aligned}$$

4. Mostrar en el archivo el proceso de factorización izquierda o justificar, en caso de no ser necesario. **(1 pt.)**

   Retomando la gramática anterior en donde eliminamos la recursividad izquierda, al inspeccionar cada una de las alternativas para todos los símbolos no terminales, se observa que no existen prefijos comunes en ninguna regla. Por ejemplo, en *tipo*, *int* y *float* comienzan con terminales distintos, *sentencia* las opciones inician con *identificador*, *if* y *while*. para *expresion* sucede lo mismo con $*$, $/$ o $\epsilon$, al igual que para *termino* con sus respectivas producciones. Dado que no existen prefijos compartidos entre producciones, la gramática ya se encuentra factorizada por la izquierda, por lo que la hace apta para la construcción directa del analizador sintáctico [1]

5. Mostrar en el archivo los nuevos conjuntos $N$ y $P$. **(0.5 pts.)**

   Por lo tanto para los nuevos conjuntos tenemos: El conjunto $N$,

$$N = \left\{ \begin{array}{l} \text{programa}, \text{declaraciones}, \text{declaraciones}', \text{declaracion}, \text{tipo}, \\ \text{lista_var}, \text{lista_var}', \text{sentencias}, \text{sentencias}', \text{sentencia}, \\ \text{expresion}, \text{expresion}', \text{termino}, \text{termino}', \text{factor} \end{array} \right\}$$

   . Finalmente para el conjunto $P$

$$P = \left\{ \begin{array}{ll} \text{programa} & \rightarrow \text{declaraciones sentencias} \\ \text{declaraciones} & \rightarrow \text{declaracion declaraciones}' \\ \text{declaraciones}' & \rightarrow \text{declaracion declaraciones}' \mid \epsilon \\ \text{declaracion} & \rightarrow \text{tipo lista_var} \; ; \\ \text{tipo} & \rightarrow \mathbf{int} \mid \mathbf{float} \\ \text{lista_var} & \rightarrow \mathbf{identificador} \; \text{lista_var}' \\ \text{lista_var}' & \rightarrow , \; \mathbf{identificador} \; \text{lista_var}' \mid \epsilon \\ \text{sentencias} & \rightarrow \text{sentencia sentencias}' \\ \text{sentencias}' & \rightarrow \text{sentencia sentencias}' \mid \epsilon \\ \text{sentencia} & \rightarrow \mathbf{identificador} = \text{expresion} \; ; \\ & \quad \mid \mathbf{if} \; ( \; \text{expresion} \; ) \; \text{sentencias} \; \mathbf{else} \; \text{sentencias} \\ & \quad \mid \mathbf{while} \; ( \; \text{expresion} \; ) \; \text{sentencias} \\ \text{expresion} & \rightarrow \text{termino expresion}' \\ \text{expresion}' & \rightarrow + \; \text{termino expresion}' \mid - \; \text{termino expresion}' \mid \epsilon \\ \text{termino} & \rightarrow \text{factor termino}' \\ \text{termino}' & \rightarrow * \; \text{factor termino}' \mid / \; \text{factor termino}' \mid \epsilon \\ \text{factor} & \rightarrow ( \; \text{expresion} \; ) \mid \mathbf{identificador} \mid \mathbf{numero} \end{array} \right.$$

6. Definir el Analizador Léxico (`lexer.ll`) con las acciones léxicas correspondientes. **(2 pts.)**

7. Implementar el Analizador Sintáctico (`Parser.cpp`) de descenso recursivo, documentando las funciones de cada No-Terminal, de forma que el programa descrito en el archivo de prueba sea reconocido y aceptado por el analizador resultante. **(4 pts.)**

### Extras

8. Documentar el código. **(0.25 pts.)**
9. Proponer 4 archivos de prueba nuevos: 2 válidos y 2 inválidos. **(0.25 pts.)**
10. Modificar el `Makefile` para poder simplificar el proceso de compilación de la práctica. **(1 pt.)**

## Referencias

[1] Dick Grune, Kees van Reeuwijk y Henri E. Bal. *Modern Compiler Design*. Second Edition. Springer.